

Technical Report

PointNeuron1.0 Reimplementation and Evaluation

Gold166 / BigNeuron Reconstruction Pipeline

Dataset	Gold166 / BigNeuron-style 3D neuron volumes
Promoted baseline	Guarded30 skeleton proposal with adaptive foreground-geodesic graph reconstruction
Main negative result	Learned connectivity GAE underperformed the geodesic initializer on the eligible validation set
Architectural boundary	Early hard samples fail even with oracle-derived nodes, showing a PointNeuron1.0 graph-stage limitation

PointNeuron Project

June 28, 2026; diagnostic closure September 21, 2026

Abstract

This report summarizes a reimplementaion and experimental evaluation of a PointNeuron-style neuron reconstruction pipeline on Gold166 / BigNeuron-style data. The project began as a replication of the PointNeuron paper pipeline: raw volume processing, point-cloud construction, skeleton proposal, graph connectivity, and final SWC reconstruction. Through iterative experiments, the work identified a stable PointNeuron1.0 baseline based on a guarded proposal checkpoint and an adaptive foreground-geodesic graph initializer. It also identified two non-promoted paths: a longer proposal checkpoint that improved validation loss but harmed end-to-end topology, and a learned connectivity GAE that underperformed the geodesic initializer. Finally, an oracle-node diagnosis showed that several early hard samples expose an architectural limitation of PointNeuron1.0: the foreground-geodesic forced-tree graph stage fails even when given GT-derived nodes. The report concludes by documenting the promoted baseline, the known limitations, and the recommended direction for PointNeuron2.0.

Contents

Executive Summary	3
1 Problem Setting and Report Scope	4
2 Purpose of the Project	4
3 Conceptual View of the Pipeline	5
3.1 PointNeuron1.0 Versus PointNeuron2.0	6
3.2 Why Negative Results Matter	6
4 Definitions and Metrics	6
4.1 Core Terms	6
4.2 Graph Terms	7
4.3 Success Criteria	8
4.4 Edge validation and biological plausibility	8
5 Repository Structure	9
6 Data Foundation	10
7 Voxel-to-Point Transformation	10
7.1 Patching within a neuron volume	10
8 Training Cache and Splits	12
9 Encoder and Proposal Model	12

10 Graph Initialization and SWC Reconstruction	13
11 Proposal Checkpoint Experiments	14
11.1 Problem: Validation Loss Was Not Enough	14
11.2 Guarded30 Checkpoint	15
12 Stable-Domain End-to-End Results	16
13 Comparison with the Original PointNeuron Paper	17
14 Early Hard Samples and Architectural Diagnosis	18
14.1 Observed Failure	18
14.2 Oracle-Node Test	18
15 Sample-Level Diagnostic Closure	19
15.1 Evidence-based review taxonomy	20
15.2 Interpretation and remaining evidence gaps	20
16 Connectivity GAE Baseline	22
17 Final PointNeuron1.0 Status	23
18 PointNeuron2.0 Implications	23
19 Reproducibility Commands	25
19.1 Environment	25
19.2 Manifest and Inspection	25
19.3 Current Baseline	25
19.4 Topology Evaluation	25
19.5 Paper-Style SWC Evaluation	25
19.6 Early Hard Oracle Diagnosis	26
19.7 Connectivity Baseline Reproduction	26
20 Conclusion	26

Executive Summary

Current state

PointNeuron1.0 has a reproducible, promoted baseline for the validated stable domain: guarded30 proposal predictions combined with adaptive foreground-geodesic graph reconstruction. This is the path to cite as the working implementation.

Main lesson

The experiments separated training loss from reconstruction quality. A longer proposal checkpoint reduced validation loss but damaged end-to-end topology, so checkpoint promotion now depends on topology evaluation rather than proposal loss alone.

Conceptual finding

PointNeuron1.0 is best understood as a hybrid system: learned local proposal of skeleton nodes followed by mostly rule-driven global graph construction. The strongest results occur when both assumptions hold. The failures occur when local node quality is not enough to rescue global connectivity.

Paper comparison

The promoted Gold166 result was also evaluated with local paper-style point-distance precision/recall/F1. It is numerically strong, but it is still not a direct leaderboard comparison to the original PointNeuron paper because the dataset and evaluator are different.

Known limit

The early hard samples are not merely undertrained proposal cases. The oracle-node diagnosis shows that the forced-tree foreground-geodesic graph stage can fail even when the node set is already strong, which makes this a PointNeuron1.0 architectural boundary and a natural target for PointNeuron2.0.

1 Problem Setting and Report Scope

Neuron reconstruction is the task of converting a 3D microscopy image of a neuron into a compact tree representation of its centerline. The input is a volumetric image: each voxel has an intensity value, and bright connected structures usually correspond to neuronal signal. The desired output is not another image. It is a graph-like skeleton whose nodes trace the neuron’s branches and whose edges describe how those branches connect.

The standard file format used here is SWC. An SWC file stores a neuronal tree as rows of node records. Each record contains a node identifier, a node type, 3D coordinates, a radius, and the identifier of its parent node. Because each node has one parent, a valid SWC describes a rooted tree. In this report, *ground-truth SWC* means the manually curated or dataset-provided reference reconstruction, while *predicted SWC* means the reconstruction exported by this project.

Gold166 is the local experimental dataset used for this work. It is treated as a BigNeuron-style collection because samples are stored as raw neuron volumes paired with SWC reconstructions and related metadata. The dataset is not perfectly uniform: volumes differ in size, intensity distribution, anisotropy, and annotation quality. Much of this project was therefore not only model training, but also making the data path reliable enough that model failures could be separated from data-loading failures.

This report is intentionally self-contained. Code paths are included so the reader can verify or reproduce the work, but the scientific and engineering argument should be understandable without opening the implementation.

2 Purpose of the Project

The purpose of this repository is to reproduce the main stages of the PointNeuron reconstruction pipeline and test how far a PointNeuron1.0-style system can be pushed on Gold166 data. The target reconstruction flow is:

```
raw 3D neuron volume
-> foreground point cloud
-> point-cloud encoder
-> skeleton proposal model
-> graph initialization / connectivity
-> SWC reconstruction
```

The original PointNeuron paper motivates a learned point-cloud approach to neuron reconstruction. The proposal document for PointNeuron2.0 then suggests improvements such as adaptive voxel-to-point sampling, a stronger PointNeXt-like point-cloud encoder, DETR-style skeleton proposal, and learned GATv2 connectivity. PointNeXt refers to a stronger modern point-cloud encoder family. DETR refers to a transformer-based detection style in which a fixed set of queries predicts objects. GATv2 refers to a graph attention network variant for learning edge- and neighborhood-aware node representations. This project does not fully implement PointNeuron2.0. Instead, it establishes a reproducible PointNeuron1.0 baseline, evaluates it, and determines which failures are fixable within the current architecture and which require a future architecture.

Conceptually, the project asks five research questions:

1. Can raw Gold166 neuron volumes and SWC labels be converted into a reliable PointNeuron-style training and evaluation pipeline?
2. Can a point-cloud proposal model learn useful candidate skeleton nodes from sparse foreground evidence?
3. Does better proposal training loss translate into better final neuron reconstruction?
4. Is a foreground-geodesic graph initializer strong enough to connect the proposed nodes into a correct SWC tree?
5. When the pipeline fails, is the failure caused by weak proposal nodes or by a deeper limitation in the graph-connectivity architecture?

3 Conceptual View of the Pipeline

The PointNeuron1.0 pipeline can be understood as a sequence of compression and decision stages. The raw image volume contains too much information to use directly as a graph. The reconstruction system must progressively compress that image into a tree while preserving the biologically important structure.

Stage	Conceptual role	Main assumption
Foreground extraction	Convert the image into candidate evidence.	Bright voxels are likely to contain neuron signal.
Point cloud encoding	Learn local geometry from sparse 3D points.	Branch shape can be inferred from neighborhoods of foreground points.
Skeleton proposal	Predict where centerline nodes should exist.	Local appearance is enough to propose useful skeleton candidates.
Graph initialization	Decide which proposed nodes connect.	True branches should have plausible foreground-geodesic paths between nearby nodes.
SWC export	Serialize the graph as a rooted neuron tree.	The final graph should be connected and tree-like; biological validity requires separate evidence.

This framing matters because not every stage has the same kind of difficulty. Proposal prediction is mostly local: it asks whether a point and its neighbors look like they belong near a centerline. Connectivity is more global: it asks which proposed nodes belong to the same branch, where branches split, and which nearby-looking structures should remain separate. A reconstruction can therefore have visually reasonable proposal nodes but still fail as a graph.

Central conceptual distinction

PointNeuron1.0 learns where skeleton nodes may be, but its strongest promoted connectivity stage is still largely rule-driven. The experiments therefore test not only a neural proposal model, but also the boundary between learned local geometry and hand-designed global topology.

3.1 PointNeuron1.0 Versus PointNeuron2.0

In this report, *PointNeuron1.0* refers to the implemented baseline: foreground point-cloud construction, DGCNN/EdgeConv proposal prediction, and foreground-geodesic graph reconstruction. It is a practical reproduction and extension of the PointNeuron-style idea, but it does not claim to solve every case in Gold166.

PointNeuron2.0 is the proposed next architecture. Its purpose is not merely to tune thresholds or add more command-line options. The motivation is conceptual: if the graph stage fails even when node locations are good, then the next system needs a stronger way to reason about edges. That points toward learned candidate-edge scoring, richer node and edge features, and topology constraints applied after edge evidence has been estimated.

3.2 Why Negative Results Matter

Several results in this report are intentionally negative. The full60 proposal checkpoint was not promoted. The first learned connectivity GAE was not promoted. The early hard samples were not folded into the first full connectivity training set. These are not abandoned experiments; they are evidence about the architecture.

The key lesson is that neuron reconstruction should be evaluated as an end-to-end structural problem. A model can look better locally while producing worse topology. A graph method can produce a valid tree while inventing long bridges. A learned edge model can achieve high recall while adding too many false connections. The report therefore treats failure modes as measurements of where the architecture’s assumptions stop matching the data.

4 Definitions and Metrics

4.1 Core Terms

The following terms are used throughout the report.

Volume A 3D image represented as a grid of voxels. In this project the volumes are Vaa3D raw files. Vaa3D is a common tool ecosystem for neuron visualization and reconstruction.

Voxel One cell of the 3D image grid. A voxel has integer coordinates and an intensity value.

Foreground The subset of voxels considered likely to belong to the neuron signal. Foreground is usually selected by intensity thresholding.

- Point cloud** A sparse set of sampled foreground voxels represented as 3D points with optional features such as intensity. PointNeuron uses point clouds so the model does not need to process every voxel in a dense 3D grid.
- SWC** The tree-format file used to represent a neuron skeleton. Each SWC node has coordinates, radius, and a parent pointer.
- Skeleton node** A point on the centerline of the neuron. In the ground truth it comes from the SWC. In prediction it comes from the proposal model or from an oracle diagnostic.
- Proposal** A candidate skeleton node predicted from the point cloud. Proposals are not final reconstructions; they still need to be filtered, selected, connected, and exported.
- Objectness** The proposal model’s confidence that a sampled point is near a real skeleton centerline.
- Center offset** A predicted displacement from a sampled foreground point toward the estimated skeleton center.
- Radius** The estimated local radius of the neuron branch near a candidate skeleton node.
- NMS** Non-maximum suppression. This removes redundant nearby proposals so that many high-confidence points on the same local structure do not all become graph nodes.
- Checkpoint** A saved model state produced during training. A checkpoint can have good validation loss while still producing poor downstream reconstructions.
- GT** Ground truth. In this report it usually means the dataset-provided reference SWC.
- Oracle nodes** Diagnostic nodes derived from ground truth rather than predicted by the model. They are used to ask: if the node locations were already good, would the graph stage still fail?

4.2 Graph Terms

- Adjacency** The edge set of a graph, often represented as a matrix where entry (i, j) indicates whether node i connects to node j .
- Geodesic path** A shortest path measured through the foreground image graph rather than by straight-line Euclidean distance. In this project, geodesic evidence asks whether two proposed nodes can be connected by a plausible bright path through the original volume.
- MST** Minimum spanning tree. Given candidate connection costs, an MST chooses a set of edges that connects all selected nodes with low total cost. The risk is that it always forces a tree, even when some connections are biologically unsupported.
- Bridge edge** A fallback edge added to connect graph components when the constrained geodesic construction cannot connect everything normally. Bridge edges are useful for producing a valid single tree, but many bridges usually indicate reliance on fallback connections. A bridge is *not* a biological neuron branch. Some bridges have reachable foreground paths but fail the normal candidate constraints; others are unreachable straight-line fallbacks. Interpret the count jointly with

bridge hit rate and reachable-edge fraction, rather than declaring every bridge false.

GAE Graph autoencoder. Here it means a neural network that receives node features and an initial graph, then predicts which node pairs should be connected.

Top-k edge selection Selecting the k highest-scoring predicted edges, usually where k is the number of target edges. This evaluates edge ranking separately from a fixed probability threshold.

4.3 Success Criteria

During the experiments it became necessary to distinguish three kinds of success:

Proposal success The proposal model predicts candidate nodes near the true skeleton.

Graph success The selected proposal nodes are connected with edges that match the topology induced by the ground-truth SWC.

Reconstruction success The graph can be converted into a valid single-root SWC that follows the neuron without adding unsupported bridges.

This distinction is important because a model can improve local proposal loss while making the final graph worse. The main topology metrics used throughout the project are:

Edge precision The fraction of predicted graph edges that match the target graph.

Edge recall The fraction of target graph edges recovered by the predicted graph.

Edge F1 The harmonic mean of edge precision and edge recall. It is useful because a graph with many false edges may have high recall but low precision, while an overly sparse graph may have high precision but low recall.

Bridge edges The number of fallback edges needed to force a connected tree. Lower is better.

Reachable edge fraction Fraction of final graph edges that were actually reachable through the foreground-geodesic image graph. Higher is better.

For evaluation, the target graph is induced from the ground-truth SWC. Predicted or selected nodes are associated with nearby ground-truth skeleton locations, and predicted edges are counted as correct when they agree with the local ground-truth connectivity implied by those assignments. This makes it possible to compare a predicted graph to the SWC tree even though the predicted nodes are not numerically identical to the ground-truth SWC nodes.

The promotion rule used in the final stage was:

A checkpoint should not be promoted unless it improves or preserves edge F1 without creating many additional bridge edges or reducing reachable edge fraction.

4.4 Edge validation and biological plausibility

Three distinct forms of evidence must be kept separate. First, *agreement with annotated morphology* is measured by `scripts/evaluate_geodesic_baseline.py`. In `src/pointneuron/graph/`

`initialization.py`, each selected proposal is assigned to its nearest annotated SWC node. Shortest-path distances along the SWC, weighted by Euclidean segment length, define a distance matrix between these assignments. An MST on that matrix supplies the GT-induced target edges. Precision, recall and F1 compare unordered proposal-node pairs with this target. This is agreement with an induced annotation-based tree, not literal equality to the original SWC edge list. Node assignment, annotation quality and MST ties can affect the target; high edge F1 does not establish spatial coverage.

Second, *image support* is measured independently of that target by the foreground-geodesic initializer. The saved graph records eligible candidate pairs, snapping distance, edge reachability, Euclidean distance and geodesic cost. The edge diagnostic script additionally reports the geodesic/Euclidean ratio, endpoint proposal scores, mean radius and continuation support. Its continuation score is the maximum absolute dot product with another incident edge direction at each endpoint, averaged over the two endpoints; it measures geometric alignment, not an independent biological label. Directions use short saved path segments where available, otherwise endpoint displacement.

The bridge/traced split in the topology evaluator permits separate hit rates. An empty bridge set is assigned hit rate 1.0 by convention; this is not empirical evidence about bridge accuracy. Unreachable edges may store a penalized fallback cost rather than a measured path length. The new table therefore also reports mean distance and ratio restricted to reachable edges. Foreground reachability can follow noise or join adjacent structures, and proposal confidence and radius are model outputs, so none is sufficient alone.

Third, *independently established biological plausibility* would require additional evidence such as expert assessment of the image and morphology, independent annotation, or validated anatomical constraints. These experiments do not provide that validation. We can say that an edge *matches the annotated morphology* and quantify image support; we cannot say it is *independently proven biologically plausible*. Target edge matching is evaluation only, and GT-derived nodes occur only in the labeled oracle test. The existing proposal script also reads SWC for input alignment checks and diagnostic coverage, but supplies no GT edges to the promoted graph inference. This closure does not change that inference path.

5 Repository Structure

This section is not required to understand the experimental argument. It is a map from the report's concepts to the repository for readers who want to inspect the implementation or rerun the experiments.

Path	Purpose
<code>configs/</code>	Local configuration templates, including paths such as the Vaa3D executable.
<code>data/</code>	Local dataset directory. Gold166 data is expected under <code>data/gold166</code> .
<code>docs/</code>	Reference PDFs, strategy notes, and this LaTeX report.
<code>scripts/</code>	Command-line tools for inspection, training, evaluation, visualization, diagnostics, and reconstruction.

<code>src/pointneuron/</code>	Importable Python implementation of data handling, models, and graph utilities.
<code>tests/</code>	Unit tests for selected model and graph behavior.
<code>tmp/</code>	Generated artifacts: manifests, splits, caches, checkpoints, E2E outputs, visualizations, and reports.

6 Data Foundation

The Gold166-style data is not a single clean tensor dataset. Each sample folder can contain raw Vaa3D volumes, several SWC files, auxiliary metadata, and occasionally invalid or misaligned labels. The first milestone was therefore a data foundation:

1. scan the dataset;
2. select one SWC per sample;
3. validate SWC structure;
4. decode raw volumes;
5. verify that SWC coordinates lie within the volume dimensions.

SWC selection priority is:

1. `*swc_sorted.swc`
2. stamped SWCs
3. base SWCs

If a preferred SWC is invalid, the scanner falls back to the next valid candidate. This prevents downstream model errors from being caused by bad data pairing.

Implementation pointers:

- `src/pointneuron/data/gold166.py`: scans the dataset and writes manifests.
- `src/pointneuron/data/swc.py`: parses and validates SWC files.
- `src/pointneuron/data/vaa3d_raw.py`: reads Vaa3D raw volumes.
- `src/pointneuron/data/alignment.py`: checks coordinate alignment.
- `scripts/build_gold166_manifest.py`: builds sample manifests.
- `scripts/check_sample_alignment.py`: verifies alignment for individual samples.

7 Voxel-to-Point Transformation

7.1 Patching within a neuron volume

The dataset unit is a whole sample volume paired with an annotated neuron SWC. It is not a patch taken across several neurons. After foreground extraction, local spatial point-cloud patches are constructed *within that same volume*. The implementation does not biologically segment separate cells before patching; noise or other foreground structures can remain in a patch.

The actual crop in `src/pointneuron/data/training_cache.py` is an axis-aligned cube, despite the parameter name `patch_radius`. Each coordinate is cropped from $\lfloor c - 96 \rfloor$ to $\lceil c + 96 + 1 \rceil$ (exclusive upper array bound), clipped to the volume boundary. Integer-centered interior patches therefore span 193 voxels per axis. Only voxels strictly above the foreground threshold are sampled, and their coordinates remain in the whole-volume coordinate frame.

The guarded30 checkpoint’s saved arguments identify the `mixed_gold_source_coverage_combined_filtered_min15_seed0` split. Its source caches record 512 points per training patch, radius 96, stride 96, coverage-based centers and a requested 32 patches per sample. The combined cache was quality-filtered, so 32 is not a claim that every sample contributed exactly 32 surviving patches. The source caches use threshold fractions 0.2 and 0.05; the filter and split are training artifacts, not settings to substitute into evaluation. These facts were checked from saved checkpoint arguments and cache manifests without loading or modifying model weights.

For full-volume inference, `scripts/run_end_to_end.py` calls `scripts/aggregate_proposals.py` with radius 96, stride 96, at most 256 patches and coverage selection. The grid spans the foreground bounding box; patches containing fewer than 256 foreground voxels are discarded. When the cap is exceeded, selection starts at the densest patch and spreads centers in normalized volume coordinates with a small density preference. Random sampling draws exactly 2048 input points per patch by the current aggregation default, using replacement when necessary. This is distinct from the 512-point training configuration. The 2048-point setting is not serialized in the proposal archives, so the diagnostic table leaves per-run points per patch unrecorded rather than treating a code default as saved evidence.

Inference uses threshold fraction 0.2, local objectness cutoff 0.5 and at most 512 local proposals per patch. Local proposals are concatenated in global coordinates, then filtered by whole-volume spherical-IoU NMS (local and global IoU thresholds 0.1), retaining at most 4096 proposals. The supplied distance radii 8 and 12 are not the suppression criterion when spherical NMS is active. This is selection/aggregation, not averaging overlapping predictions. Saved `patches` records the actual per-sample count (3–220 in the completed promoted runs). Graph selection then applies the separate promoted configuration: score 0.85, adaptive connected coverage, distance NMS 18 and maximum 128 nodes. Graph foreground extraction starts at intensity 5 and adapts toward a 350,000-voxel cap; it is separate from the proposal threshold and can remain over cap when even the brightest nonempty foreground is too large.

The first PointNeuron-style transformation converts the dense image volume into a point cloud:

```
voxels with intensity > threshold  
-> (x, y, z, intensity)
```

This is intentionally close to the original PointNeuron idea: use a point-cloud network rather than process the full voxel grid directly. The advantage is that neuron volumes are sparse and large. The weakness is that a fixed threshold is not equally reliable across all samples. Some volumes are noisy, saturated, or anisotropic. These issues later become important for the early hard samples.

In practical terms, thresholding decides which voxels are allowed to become evidence. If the threshold is too low, the point cloud includes background noise and false structures. If it is too high, faint but real branches disappear. The early hard samples expose this tension because their intensity distributions are less well behaved than the stable validation range.

Implementation pointers:

- `src/pointneuron/data/point_cloud.py`: converts volumes to point clouds.
- `scripts/build_point_cloud.py`: writes inspectable point-cloud CSV files.
- `scripts/visualize_sample.py`: renders foreground points and SWC overlays to HTML.

8 Training Cache and Splits

The raw volumes are too large for direct proposal training. The project builds patch-level training records that contain foreground points and nearby SWC skeleton nodes. These records are stored as `.npz` files and then split deterministically into train/validation/test groups.

A patch is a smaller spatial crop sampled from a full volume. Patch training is used because full 3D neuron volumes can contain far more foreground points than fit comfortably into a single training example. Deterministic splits are used so that the same records remain in the same train/validation/test groups across runs, making checkpoint comparisons meaningful.

Each cache record contains:

```
sampld foreground points
SWC skeleton nodes in or near the patch
edge indices and skeleton metadata
sample and patch metadata
```

Implementation pointers:

- `src/pointneuron/data/training_cache.py`: creates cached training records.
- `src/pointneuron/data/torch_dataset.py`: loads cached records for PyTorch training.
- `src/pointneuron/data/splits.py`: builds deterministic splits.
- `scripts/build_training_cache.py`: generates training caches.
- `scripts/build_split.py`: writes split files.
- `scripts/inspect_dataset.py`: checks loader and tensor shapes.

9 Encoder and Proposal Model

The proposal stage implements a DGCNN/EdgeConv-style encoder and a proposal head that predicts:

Objectness whether a point is near the skeleton.

Center offset how to move from the sampled point toward a skeleton center.

Radius local structural radius.

This stage corresponds to the skeleton proposal component of PointNeuron. It does not directly reconstruct a neuron. It produces a large set of candidate skeleton nodes that must later be selected and connected.

DGCNN means Dynamic Graph Convolutional Neural Network. Its EdgeConv operation builds local neighborhoods in point space and learns features from relationships between neighboring points. This is appropriate for neuron point clouds because branch geometry is local: nearby foreground points often carry information about centerline direction and radius.

Implementation pointers:

- `src/pointneuron/models/dgcnn.py`: DGCNN/EdgeConv encoder.
- `src/pointneuron/models/proposal.py`: proposal head.
- `src/pointneuron/models/proposal_loss.py`: proposal losses.
- `scripts/train_proposal.py`: trains proposal checkpoints.
- `scripts/audit_proposal_checkpoint.py`: audits proposal behavior.
- `scripts/aggregate_proposals.py`: applies a checkpoint to a whole volume.
- `scripts/visualize_proposals.py`: visualizes proposals.

10 Graph Initialization and SWC Reconstruction

The strongest PointNeuron1.0 graph path is the foreground-geodesic initializer. Its basic assumption is:

Two proposal nodes should be connected if there is a plausible bright foreground path between them through the original image volume.

The graph initializer:

1. filters proposal nodes by confidence and non-maximum suppression;
2. keeps a limited graph-node budget;
3. snaps selected nodes to foreground voxels;
4. computes shortest paths through foreground voxels;
5. builds an MST-like graph using geodesic distances;
6. adds bridge edges when needed to connect components;
7. exports edge paths to SWC.

The phrase *adaptive connected coverage NMS* refers to the promoted node selection strategy. It begins from high-confidence proposals, suppresses nearby duplicates, and tries to preserve spatial coverage so the selected nodes are not all concentrated in one bright region. The word *connected* reflects that selection is judged by whether the later geodesic graph can plausibly connect the selected nodes.

Implementation pointers:

- `src/pointneuron/graph/initialization.py`: shared graph utilities.
- `scripts/initialize_geodesic_graph.py`: foreground-geodesic graph construction.
- `scripts/generate_swc_from_graph.py`: graph-to-SWC export.
- `scripts/visualize_reconstruction.py`: reconstruction HTML visualization.
- `scripts/run_end_to_end.py`: full proposal-to-SWC pipeline.
- `scripts/evaluate_geodesic_baseline.py`: topology evaluation.
- `scripts/evaluate_paper_style_swc.py`: point-distance SWC evaluation for paper-style precision, recall, F1, and approximate ESA/DSA/PDS.

11 Proposal Checkpoint Experiments

The proposal checkpoint names encode how a model was trained. For example, `full60` means a longer 60-epoch training run, while `guarded30` means a 30-epoch run with more conservative losses intended to prevent the proposal head from drifting too aggressively. These names are project labels, not external benchmarks.

11.1 Problem: Validation Loss Was Not Enough

Early proposal checkpoints were evaluated with proposal-level losses and metrics. This was insufficient. A checkpoint can reduce validation loss while increasing center drift, increasing confidence, and harming final graph topology.

The failed `full60` checkpoint illustrates this:

```
tmp/checkpoints/proposal_conservative_rawcenter_full60_cw3_oreg001_nw1.pt
```

It reached a better validation loss, approximately:

```
val_loss: 5.0645
```

but harmed end-to-end topology on the hard probe:

Checkpoint	F1	Bridges	Reachable
old rawcenter	0.3937	15.25	0.8878
full60	0.4232	40.25	0.6831

Although F1 rose slightly, bridges increased dramatically and reachable fraction collapsed. Therefore full60 was not promoted.

Promotion rule

Proposal checkpoints are not promoted by validation loss alone. A checkpoint must preserve usable graph topology: low bridge count, high reachable fraction, and stable reconstruction behavior after SWC export.

11.2 Guarded30 Checkpoint

The promoted proposal checkpoint is:

```
tmp/checkpoints/proposal_conservative_guarded30_cw3_oreg005_nw3.pt
```

It was trained with conservative losses and stronger regularization:

```
epochs: 30
batch_size: 2
lr: 0.0001
proposal_coordinate_mode: raw
loss_mode: conservative
conservative_center_weight: 3.0
offset_regularization_weight: 0.005
non_worsen_weight: 3.0
augment: True
amp: True
best_epoch: 27
best_val_loss: 5.215460332957181
```

Proposal audit:

In the audit table, *proposal hit* measures how often predicted candidate nodes land near target skeleton structure. *Offset mean* is the average size of the predicted center displacement, so a large increase can indicate that the model is making more aggressive moves away from the sampled foreground points. *Score P90* is the 90th percentile proposal confidence; high confidence is only useful if the resulting nodes remain topologically helpful.

Checkpoint	Proposal Hit	Offset Mean	Score P90
old rawcenter	0.3897	0.5422	0.8489
full60	0.3980	0.9701	0.9265
guarded30	0.3910	0.6258	0.8616

Guarded30 did not dramatically improve local proposal metrics, but it preserved controlled offsets and gave better graph behavior.

Promoted proposal checkpoint

Guarded30 is the PointNeuron1.0 proposal checkpoint because it trades a small amount of local proposal ambition for more reliable downstream topology.

12 Stable-Domain End-to-End Results

The main stable validation range was samples 100–162. This range is called *stable* because the data path, foreground extraction, proposal selection, and geodesic graph construction produced interpretable reconstructions without the extreme bridge failures seen in the early hard samples. It is not claimed to be the entire problem solved; it is the validated domain where PointNeuron1.0 can be reported honestly.

The promoted graph configuration was:

```
initializer: geodesic
graph selection: adaptive_connected_coverage_nms
min proposal score: 0.85
NMS mode: distance
NMS distance: 18
max graph nodes: 128
```

The label *old adaptive v4* refers to the previous best adaptive geodesic baseline before guarded30 was promoted. It is included as a control so the reader can see whether the new proposal checkpoint improved the full reconstruction pipeline rather than only improving an isolated training loss.

Range	Method	F1	Bridges	Reachable
100–109	old adaptive v4	0.7822	1.20	0.9659
100–109	guarded30	0.7868	1.00	0.9363
110–129	old adaptive v4	0.7545	1.05	0.9667
110–129	guarded30	0.7827	0.65	0.9798
130–162	old adaptive v4	0.8580	2.1515	0.9308
130–162	guarded30	0.8708	2.0606	0.9368

Combined stable-domain comparison:

Method	F1	Bridges	Reachable
old adaptive v4, 100–162	0.8131	1.651	0.9478
guarded30 adaptive v4, 100–162	approx. 0.8295	approx. 1.444	approx. 0.9504

Conclusion: guarded30 is promoted for PointNeuron1.0 stable-domain proposals.

Stable-domain result

Across samples 100–162, guarded30 with adaptive geodesic reconstruction gives the best validated PointNeuron1.0 behavior: higher F1, fewer bridges, and a slightly higher reachable fraction than the previous adaptive baseline.

13 Comparison with the Original PointNeuron Paper

The original PointNeuron paper reports results on the Janelia-Fly dataset from the BigNeuron project, not on Gold166. Its test protocol uses 42 total images, split into 35 training images, 3 validation images, and 4 testing images. It reports spatial reconstruction metrics computed through Vaa3D-style comparison: ESA, DSA, PDS, precision, recall, and F1.

To make the comparison more meaningful, this project now includes a local paper-style SWC evaluator:

```
scripts/evaluate_paper_style_swc.py
```

The evaluator resamples the predicted and ground-truth SWC trees into 3D point sets, then computes point-distance precision, recall, and F1 at a 6-voxel threshold. It also reports local approximations of ESA, DSA, and PDS. These approximate ESA/DSA/PDS values are useful for internal comparison, but they are not claimed to be byte-identical to the official Vaa3D metric plugin used by the paper.

The paper’s strongest APP2-initialized PointNeuron result is:

Method from paper	Precision	Recall	F1
APP2 baseline	64.51	51.67	52.63
APP2 + PointNeuron	91.56	63.17	71.70

The same table reports that the APP2 + PointNeuron row achieved $ESA = 1.43$, $DSA = 4.13$, and $PDS = 0.18$. These are paper-style geometric discrepancy metrics; lower is better.

Our strongest promoted result, evaluated on the Gold166 stable domain with the new local paper-style evaluator, is:

This project	Precision	Recall	F1
Guarded30 adaptive geodesic, Gold166 stable domain	96.99	71.95	81.50

The same local evaluator reports:

This project	Approx. ESA	Approx. DSA	Approx. PDS
Guarded30, stable Gold166	3.6774	5.9063	0.1553

This result is stronger than the original edge/topology-only comparison because it evaluates the exported SWC geometry directly. It says that, on the validated Gold166 stable domain, the promoted reconstruction is spatially close to the ground truth with high precision and good recall.

However, this still should not be reported as a formal claim that the project beats the PointNeuron paper. The F1 value is now closer in spirit to the paper’s F1 than the earlier edge F1 was, but the dataset is still different and the ESA/DSA/PDS values are local approximations rather than official Vaa3D plugin outputs.

Paper comparison conclusion

Against the paper’s reported APP2 + PointNeuron F1 of 71.70%, our promoted Gold166 stable-domain run obtains 81.50% local point-distance F1. This is a strong result, but it is contextual rather than a direct published-benchmark win. A formal claim would require running the same evaluator protocol on a paper-compatible Janelia-Fly split or running the official Vaa3D metric plugin on both outputs.

This distinction is important. The current report establishes that the reimplement is functional and that the promoted PointNeuron1.0 baseline is reasonable on its validated domain. It does not establish a direct leaderboard comparison against the original PointNeuron paper.

14 Early Hard Samples and Architectural Diagnosis

14.1 Observed Failure

The early hard samples are samples in the 0–24 range that repeatedly produced unreliable graph topology. They are difficult because the volume geometry and foreground signal can be unusually large, saturated, noisy, anisotropic, or otherwise hostile to the foreground-geodesic assumption. On this range, guarded30 improved some local behavior but graph topology became unstable:

```
guarded30 000-024:
  samples 22
  F1 0.4709
  bridges 26.9091
  reachable 0.8049
```

The worst samples were:

```
7, 16, 22, 23, 24
```

Score-NMS on these reused proposals made the situation worse:

```
score_nms early worst:
  F1 0.3075
  bridges 115.4
  reachable 0.1134
```

This showed that the adaptive selector was not the primary problem. It was partially protecting the graph stage.

14.2 Oracle-Node Test

To distinguish proposal failure from architecture failure, the project added:

```
scripts/diagnose_beast_oracle.py
```

This script replaces learned proposal nodes with GT-derived oracle nodes and then runs the same foreground-geodesic initializer. If the graph stage works with oracle nodes, the problem is

proposal-side. If it still fails, the graph architecture itself is insufficient.

Oracle result:

```
oracle early hard samples:
  mean F1 0.5889
  mean bridges 76.6000
  mean reachable 0.3969
```

Sample	F1	Bridges	Reachable
7	0.2677	124	0.0236
16	0.5433	104	0.1811
22	0.8898	0	1.0000
23	0.6373	73	0.4252
24	0.6063	82	0.3543

Samples 7, 16, 23 and 24 retain poor foreground connectivity with oracle nodes. This establishes a graph-stage limitation under the existing preprocessing and forced-tree rules, without proving that graph architecture is the only cause. Sample 22 is an important counterexample: its oracle graph has F1 0.8898, zero bridges and full reachability. Its learned-node failure is therefore sensitive to upstream node placement/selection. A supported failure pathway for several of the other cases is:

```
saturated / large / anisotropic volume (candidate contributing factors)
-> foreground threshold adaptation, sometimes still over cap
-> eligible geodesic pairs collapse
-> MST still forces one connected tree
-> fake long bridge edges
```

The relative contributions of intensity saturation, geometry and annotation alignment are not isolated by this experiment. These cases remain diagnostic limitations of PointNeuron1.0; no PointNeuron2.0 work is initiated here.

Architectural diagnosis

The early hard samples are not being avoided because they are inconvenient. They are separated because the oracle-node test shows a real graph-stage failure mode: the current PointNeuron1.0 geodesic tree builder can invent long bridges even when the node candidates are already near the ground truth.

15 Sample-Level Diagnostic Closure

The post-meeting analysis is generated by `scripts/build_sample_diagnostics.py` from saved experiment artifacts. The complete table is `tmp/diagnostics/pointneuron1_sample_diagnostics.csv`; its JSON companion retains original observations, per-field provenance and separate historical, oracle and GAE evidence. Exact input paths are listed in `tmp/diagnostics/pointneuron1_`

`sources.json`. The mapping and review rules are documented in `docs/sample_diagnostics.md`. No model weights, training procedure, graph settings or existing experiment results were changed.

The local inventory contains 106 evaluated/requested sample identities: 88 promoted requests (0–24 and 100–162), 13 with historical primary runs and five with auxiliary evidence only. All 88 promoted requests are represented, including the three recorded execution failures, samples 3, 4 and 6. Forty-three rows lack at least one core or paper-style field; nine lack canonical reconstruction/topology evidence. Paper-style geometry is available for the 63 promoted stable-domain samples. Missing values are NA/null, not zero.

Each row describes one primary run. Promoted identity is checked against graph metadata, rather than inferred solely from an experiment directory name. Selection audits join by graph path and spatial metrics by exported SWC path. Older results remain source-qualified and cannot fill missing promoted metrics. The selection audit first score-filters proposals; its candidate count and branch coverage are therefore kept separate from aggregation metrics, including their different empty-branch conventions. The large source inventory includes historical controls; its size must not be mistaken for independent sample count.

15.1 Evidence-based review taxonomy

`execution_status` records completed versus failed runs. `run_status` instead records diagnostic triage: PASS means no trigger in the available evidence; FAIL means review is required, including unresolved or incomplete cases. Neither is a biological acceptance criterion. In this pass, 52 samples have no trigger and 54 require review: six proposal, five selection, four data-preprocessing, five mixed and 34 unknown. There are no exclusively connectivity-graph or output-geometry assignments; oracle-supported graph deficits coexist with other evidence and are counted as mixed.

The conservative rules attribute an unsatisfied foreground cap to preprocessing; candidate coverage below 0.50 to proposal-side evidence; and candidate segment coverage at least 0.70 followed by loss at least 0.35 to selection-side evidence. Oracle reachability below 0.50 with more than five bridges supports a graph-stage limitation. Invalid output, or edge F1 at least 0.80 with spatial F1 below 0.50, supports an output discrepancy. Multiple supported stages are labeled mixed. Graph F1 below 0.65, reachability below 0.90 or more than five bridges triggers review but does not by itself identify a cause. These last cutoffs reuse the existing connectivity-eligibility thresholds; they are review heuristics, not validated performance boundaries. Ambiguous cases remain unknown and all flagged rows require manual review.

15.2 Interpretation and remaining evidence gaps

The stable-domain guarded30 runs have mean bridge count about 1.444, compared with 26.909 in the 22 completed early hard runs. The five oracle cases average 76.6 bridges, but are a deliberately difficult subset, not a population-matched comparison. Sample 19 shows why low bridge count alone is insufficient: selected nodes lose much of the annotated skeleton before graph construction. Sample 22 shows that the successful oracle intervention can reverse the diagnosis implied by the group mean. Samples 7, 16, 23 and 24 demonstrate that an oracle node set does not remove the foreground-connectivity limitation.

The saved data audits support an alignment-guard explanation for 3, 4 and 6, but do not preserve their subprocess traces or explain the underlying annotation misalignment. They do not independently validate biological plausibility or isolate which image property causes each graph failure. Stable-

Sample	Attribution	Recorded evidence and interpretation
3, 4, 6	Preprocessing	Saved data audits record 17, 25 and 24 out-of-bounds SWC nodes. Aggregation’s alignment guard exits with the recorded code 2; subprocess output itself is unavailable.
7	Mixed	Candidate segment coverage 0.107; graph foreground cap unsatisfied. Oracle nodes still give 124 bridges and reachability 0.024.
16	Mixed	Foreground cap unsatisfied. Oracle graph retains 104 bridges and reachability 0.181; proposal improvement alone cannot explain away this deficit.
19	Selection	Candidate segment coverage 0.986 falls to 0.409 after selection; edge F1 0.472 despite reachability 0.992 and only one bridge.
22	Proposal	Candidate coverage is zero in the saved audit; learned-node edge F1 0.110. Oracle nodes recover F1 0.890 with no bridges and full reachability.
23	Mixed	Candidate segment coverage 0.225; learned graph has 127 bridges and zero reachability. Oracle graph still needs 73 bridges (reachability 0.425).
24	Mixed	Candidate segment coverage 0.278 and unsatisfied foreground cap. Oracle graph still needs 82 bridges (reachability 0.354).
105	Unknown	Spatial F1 0.388 and edge F1 0.600 with no bridges and full reachability. A matched promoted selection audit is absent; a unique cause cannot be assigned.

Table 2: Representative failures and review cases. Values are from the promoted run unless explicitly labeled oracle. Full records and evidence are in the CSV/JSON.

domain selection audits from older checkpoints cannot be substituted for missing promoted audits. Obtaining missing spatial metrics or matched selection audits would require additional lightweight evaluation of saved outputs, not retraining. Confirming the exact failed subprocess traces would require locating further logs or a controlled rerun with output capture. Separating thresholding, node placement and graph-rule effects requires new controlled experiments. Independent biological review and an external-dataset test require new evidence. The schema includes dataset identity and retains missingness for future reuse, but no Allen ingestion or PointNeuron2.0 work is part of this closure. Further development is deferred until this review.

16 Connectivity GAE Baseline

The repository includes a PointNeuron-style graph autoencoder:

```
src/pointneuron/models/connectivity.py
```

The GAE receives node features and an initial adjacency, then predicts an adjacency matrix. In other words, it tries to learn which selected skeleton nodes should be connected after seeing both node-level information and the initial geodesic graph.

It was trained only on graph-eligible samples to avoid learning from corrupted bridge-heavy targets. A sample was considered eligible when the initializer already produced enough usable graph signal: sufficient nodes, acceptable edge F1, acceptable reachable fraction, and no foreground-cap failure. A foreground-cap failure means the foreground extraction produced a point set that violated the practical limits used by the graph construction path, usually because the sample's intensity/size behavior made the foreground too large or unstable to treat as normal evidence. This was a training-control decision. It prevented the first GAE baseline from being dominated by cases where the target graph itself was already contaminated by obvious reconstruction failure.

Eligibility result:

```
accepted records: 46
rejected records: 39

rejection reasons:
  foreground_cap_not_satisfied: 5
  low_edge_f1: 17
  low_reachable: 13
  too_few_nodes: 4
```

Training result:

```
records: 46
best_epoch: 46
best_loss: 0.377377
checkpoint: tmp/checkpoints/connectivity_guarded30_eligible_50e.pt
```

Evaluation result:

```
threshold_precision mean 0.3702
threshold_recall mean 0.9490
```

```
threshold_f1 mean 0.4983
topk_f1 mean 0.5200
init_topk_f1 mean 0.8261
```

The threshold metrics use a fixed probability cutoff of 0.5. The top-k metrics instead select the highest-scoring predicted edges until the same number of edges as the target graph is reached. The comparison to `init_topk_f1` asks whether the learned GAE improves over the graph it was given as input. In this run it did not.

The saved evaluation covers the eligible records used for this baseline; these figures are not evidence of an independent external-dataset test.

Conclusion: the GAE learned a high-recall, low-precision edge signal. It did not beat the initialized geodesic graph and is not promoted.

Connectivity decision

The learned GAE is useful as a baseline and a diagnostic artifact, but not as the deployed PointNeuron1.0 connectivity stage. Its precision is too low, and its top-k F1 remains below the initializer it was meant to improve.

17 Final PointNeuron1.0 Status

Promoted:

```
proposal checkpoint:
  tmp/checkpoints/proposal_conservative_guarded30_cw3_oreg005_nw3.pt

topology path:
  adaptive_connected_coverage_nms geodesic graph
```

Not promoted:

```
tmp/checkpoints/proposal_conservative_rawcenter_full60_cw3_oreg001_nw1.pt
tmp/checkpoints/connectivity_guarded30_eligible_50e.pt
```

Boundary:

```
PointNeuron1.0 is suitable for the stable validated Gold166 subset.
PointNeuron1.0 is not architecture-complete for the early hard samples.
The first connectivity GAE does not replace the geodesic initializer.
```

18 PointNeuron2.0 Implications

This section preserves prospective discussion from the earlier report. It is not an implementation plan authorized by the diagnostic closure; PointNeuron2.0 remains deferred pending review of the sample-level evidence above.

The PointNeuron2.0 proposal is aligned with the observed failure, but the most important fix is learned connectivity. Better proposals alone are not enough, because the oracle-node experiment showed that even GT-derived nodes fail under the current foreground-geodesic forced-tree rule.

Recommended PointNeuron2.0 direction:

1. Use stronger point-cloud encoding and proposal generation, such as the proposed PointNeXt / transformer-decoder detection approach.
2. Build multi-scale candidate edges rather than scoring all possible pairs equally.
3. Train an edge classifier or reranker using learned features, Euclidean geometry, radius information, local direction, and foreground-geodesic evidence.
4. Treat foreground geodesic as a feature, not the final source of truth.
5. Apply topology constraints after edge scoring rather than forcing a tree before edge evidence is reliable.

19 Reproducibility Commands

19.1 Environment

```
py scripts\check_environment.py --config configs\local.json
py -m pip install -e .
py -m pip install scipy
```

19.2 Manifest and Inspection

```
py scripts\build_gold166_manifest.py '
--root data\gold166 '
--output tmp\gold166_manifest.json

py scripts\inspect_volume.py --sample-index 0 --decode

py scripts\check_sample_alignment.py '
--sample-index 0 '
--decode-volume
```

19.3 Current Baseline

```
py scripts\run_end_to_end.py '
--sample-index 110 '
--output-root tmp\e2e_guarded30_single '
--checkpoint tmp\checkpoints\proposal_conservative_guarded30_cw3_oreg005_nw3.pt '
--initializer geodesic '
--graph-selection-mode adaptive_connected_coverage_nms '
--graph-min-proposal-score 0.85 '
--graph-nms-mode distance '
--graph-nms-distance 18 '
--graph-max-nodes 128 '
--device cuda
```

19.4 Topology Evaluation

```
py scripts\evaluate_geodesic_baseline.py '
--summary tmp\e2e_guarded30_adaptive_v4_110_129\summary.json '
--csv-output tmp\e2e_guarded30_adaptive_v4_110_129\topology_report.csv '
--json-output tmp\e2e_guarded30_adaptive_v4_110_129\topology_report.json
```

19.5 Paper-Style SWC Evaluation

```
py scripts\evaluate_paper_style_swc.py '
--summary tmp\e2e_guarded30_adaptive_v4_100_109\summary.json '
```

```
--summary tmp\e2e_guarded30_adaptive_v4_110_129\summary.json '
--summary tmp\e2e_guarded30_adaptive_v4_130_162\summary.json '
--csv-output tmp\paper_style_eval\guarded30_stable_100_162.csv '
--json-output tmp\paper_style_eval\guarded30_stable_100_162.json
```

19.6 Early Hard Oracle Diagnosis

```
py scripts\diagnose_beast_oracle.py '
--output-root tmp\beast_oracle_diagnosis
```

19.7 Connectivity Baseline Reproduction

```
py scripts\build_eligible_connectivity_manifest.py '
--summary tmp\e2e_guarded30_adaptive_v4_000_024\summary.json '
--summary tmp\e2e_guarded30_adaptive_v4_100_109\summary.json '
--summary tmp\e2e_guarded30_adaptive_v4_110_129\summary.json '
--summary tmp\e2e_guarded30_adaptive_v4_130_162\summary.json '
--output-root tmp\connectivity_guarded30_eligible '
--include-score '
--skip-existing

py scripts\train_connectivity.py '
--record-manifest tmp\connectivity_guarded30_eligible\eligible_connectivity_manifest.
  json '
--epochs 50 '
--lr 1e-3 '
--weight-decay 1e-4 '
--normalize-node-features '
--init-decoder-bias '
--checkpoint tmp\checkpoints\connectivity_guarded30_eligible_50e.pt '
--device cuda
```

20 Conclusion

The project successfully reproduced the main PointNeuron1.0 pipeline stages and produced a stable guarded30 + adaptive geodesic baseline for the validated domain. On the Gold166 stable domain, this baseline reaches approximately 81.50% local point-distance F1 with high precision and good recall under the new paper-style SWC evaluator. It also clarified two important negative results. First, lower proposal validation loss does not necessarily imply better reconstruction. Second, the current full-adjacency connectivity GAE does not outperform the geodesic graph initializer.

The diagnostic closure preserves the oracle-supported graph limitation while distinguishing sample 22, which recovers with oracle nodes, and selection losses such as sample 19. The full per-sample table exposes unresolved causes and missing metrics. Agreement with annotated morphology remains distinct from independent biological validation. PointNeuron2.0 work is deferred until this closure is reviewed.